

3C7 Digital Systems Design

Assignment 1

Name: Jing Yi Yang 23376344

Introduction

The purpose of this assignment is to design and implement a Mini Arithmetic Logic Unit (Mini-ALU) using Verilog and deploy it on the Basys-3 FPGA board.

The arithmetic logic unit (ALU) does several different arithmetic and logical operations on the two 6-bit inputs, called A and B, and these numbers are represented in 2's complement format.

The output of the arithmetic logic unit is a 6-bit result X, which is determined by the value of the function selector input (fxn).

fxn	X[5:0]
000	A
001	B
010	-A
011	-B
100	$A < B$ (is A less than B)
101	$(A \text{ } \textit{nxor} \text{ } B)$ (Bitwise XNOR)
110	A+B
111	A-B

Figure 1: ALU operation control table

The design was implemented using modules created in Lab C, in particular, the six-bit ripple carry adder, and then integrated into a hierarchical ALU structure as requested in this assignment.

Design Strategy

The arithmetic logic unit is designed using a **modular hierarchical approach**. This means the system was divided into smaller modules that each performed its own a specific task. These modules were then connected together to build the final completed ALU system.

The following modules were used in my design:

module	Purpose
FullAdder.v	Performs 1-bit addition
Sixbit_ripple_adder.v	Performs 6-bit addition/subtraction
Mini_alu.v	Implement the ALU operations
Alu_top.v	The top level module that connects the design to the FPGA board

The six-bit ripple carry adder from Lab C was reused to perform both addition and subtraction.

Subtraction was implemented using two's complement arithmetic, where the input operand is inverted and a carry-in of 1 is added.

The ALU module then selects the correct output operation based on the value of the fxn input.

Design Hierarchy

The overall system is organised in a hierarchical structure as shown below:

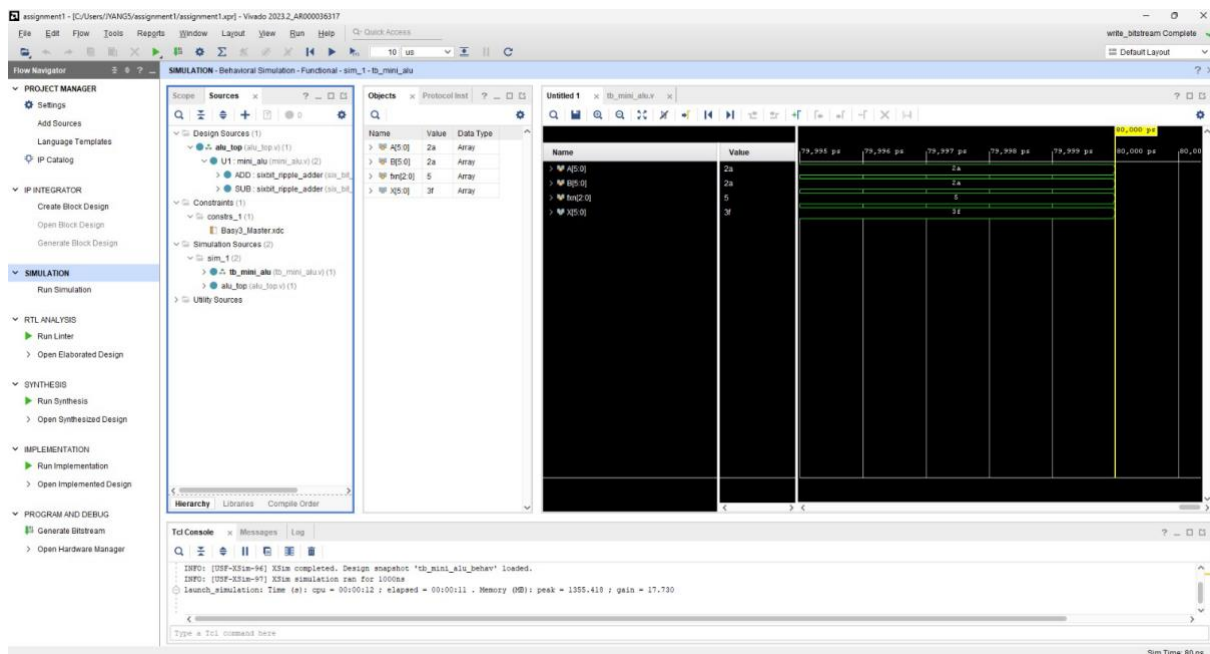


Figure 2: screenshot of the design hierarchy.

This makes the design easier to understand and will allow each module to be tested separately.

Modules Reused and Adapted

Two of the modules I've used in the assignment was from previous lab, LabC:

FullAdder

The FullAdder module was reused from LabC and it performs the basic 1-bit addition operation.

Six-Bit Ripple Carry Adder

The six-bit ripple carry adder was also reused from Lab C, and it forms the main arithmetic unit of the ALU. Which it supports both addition and subtraction operations through the use of a control input (sel).

Mini ALU Module

The mini_alu module was created to combine the outputs of different operations and select the correct output using a case statement based on the fxn input.

Top Level Module

This module is the top-level design. It contains only module instantiations and connects the ALU inputs and the outputs to the FPGA board switches and also LEDs..

Testbench and Testing Strategy

A testbench called tb_mini_alu was created to test my ALU design.

This testbench applies different values for A, B and fxn to make sure that each operation will work correctly.

The first test vector used the last six bits of the board number, as required in the assignment handout given to us.

Additional test vectors were then used to verify each ALU operation.

Example test cases included:

A	B	fxn	Expected Result
000011	000010	110	000101
000101	000010	111	000011
000001	000010	100	000001
101010	101010	101	111111

Waveform Results

The simulation waveforms confirm that the ALU produces the correct outputs for each operation.

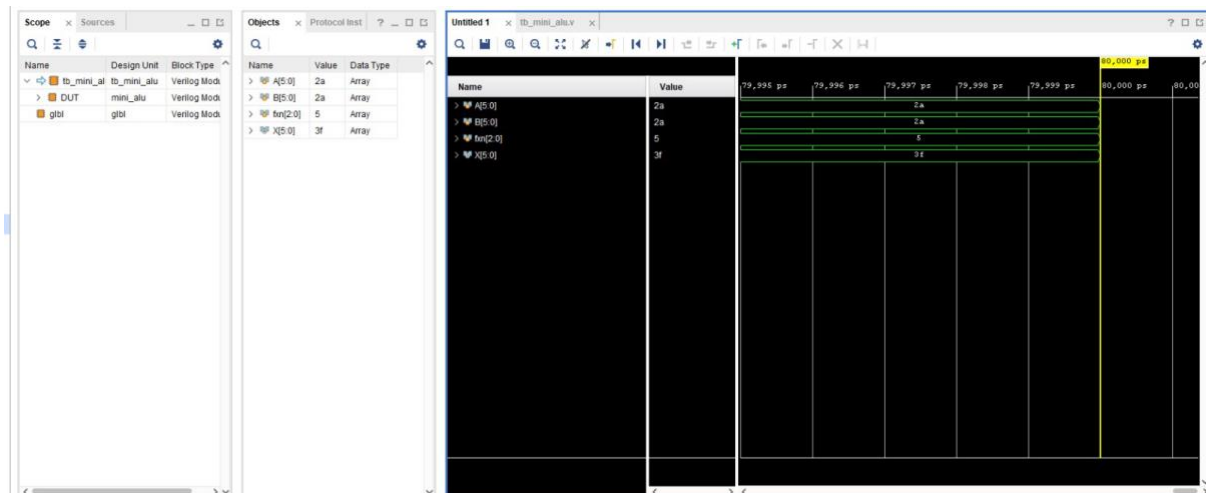


figure 3: screenshot of the waveforms.

The waveform shows the inputs A, B, and fxn changing over time, with the output X updating accordingly. The results match the expected outputs for each ALU function

Testing Evaluation

Each ALU function was tested through simulation to confirm correct operation.

The test strategy covered all arithmetic and logical operations defined in the specification. While the selected test vectors demonstrate correct behaviour for common cases, additional testing with more varied inputs could further increase confidence in the design.

Robustness of the Design

The ALU is designed to handle all valid 6-bit inputs using two's complement representation. Because the operations are implemented using combinational logic, the design produces outputs immediately when the inputs change.

Unexpected input combinations will still produce valid outputs due to the deterministic nature of the combinational logic.

Hardware Implementation

After successful simulation, the design was synthesized and implemented using Vivado. A bitstream file was generated and downloaded to the Basys-3 FPGA board.

The FPGA board switches were used to control the ALU inputs, and the LEDs were used to display the output result.

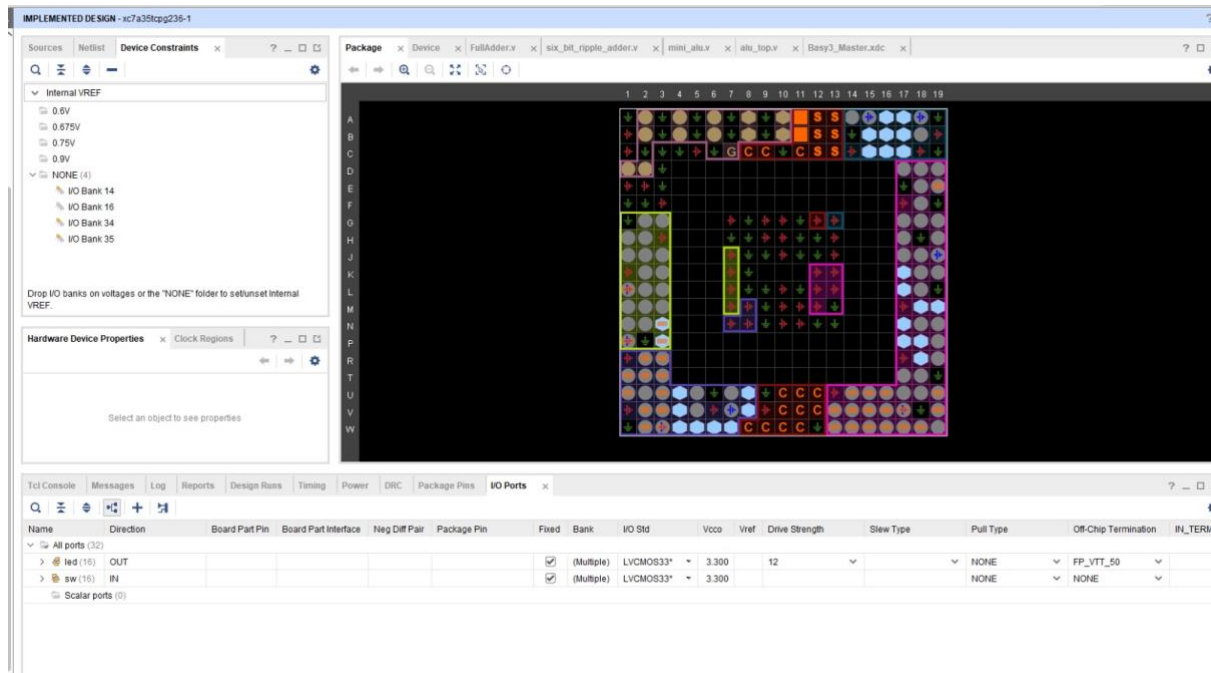


Figure 4: FPGA device view, which shows implemented design and I/O port mapping in vivado

The figure above shows the implemented design on the FPGA device, which confirms that the switch inputs and LED outputs are correctly connected through the design constraints file.

The resource utilization report was also generated and shows the hardware resources used by the mini alu implementation.

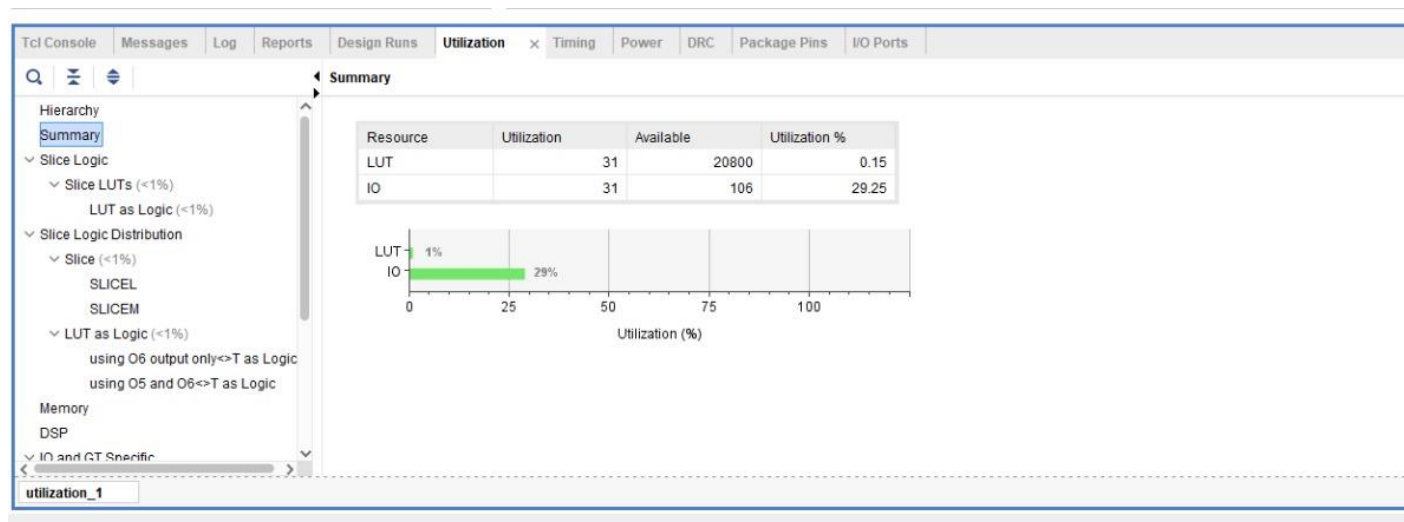


Figure 5: resource utilization summary

From this report, the design uses a small portion of the available FPGA. Only 31 LUTs out of 20800 available LUTs were used, which is approx. 0.15 percent utilization. It also used 31 I/O pins out of the 106 available pins, which is approx 29 percent utilization.

This report shows that the mini alu design is lightweight and efficient, in terms of hardware resources.

Vivado also performs a timing analysis to make sure that the design meets timing requirements.

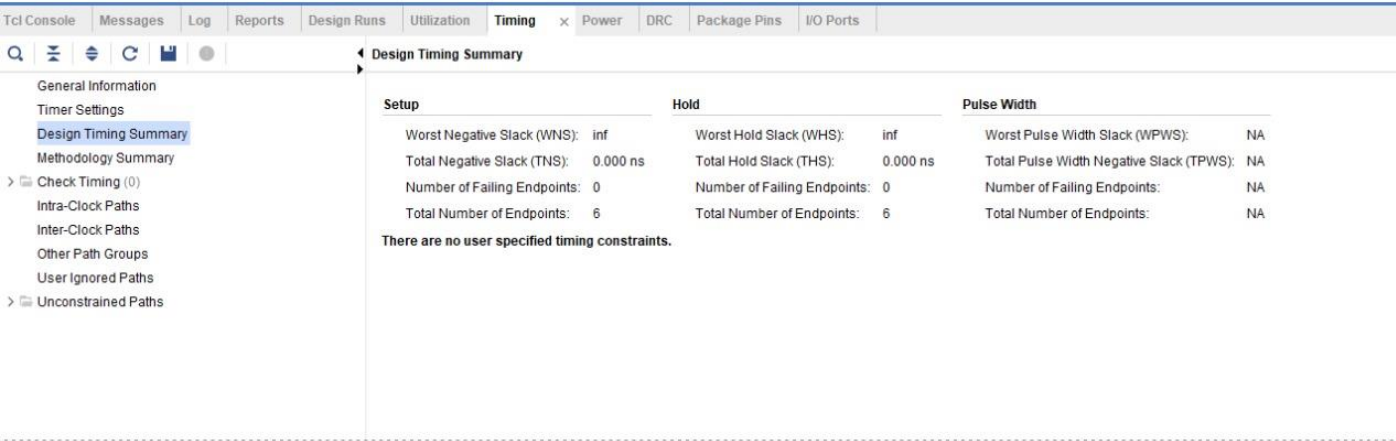


Figure 6: timing summary report

The timing summary report shows that there are no failing timing paths in the design. The worst negative slack and total negative slack are both 0 ns, which means that the design meets the required timing constraints. This confirms that the mini alu can operate correctly on the FPGA hardware.

Demo

The Mini-ALU was demonstrated on the Basys-3 board using the following switch configuration.

Switch	Function
SW0-SW5	Input A
SW6-SW11	Input B
SW12-SW14	Function selector(fxn

LED	Output
LED0-LED5	ALU result X

Example demonstrations on board:

Test 1:

A	B	fxn	Operation	Result
000011	000010	110	A+B	000101

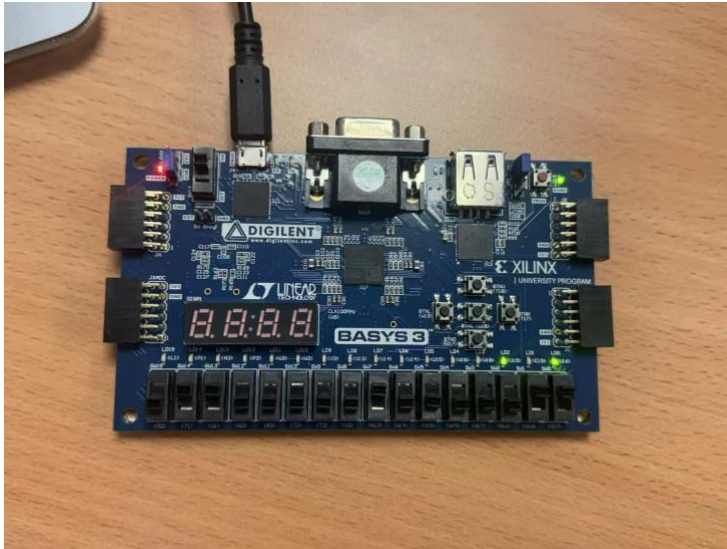


Figure 7: test 1

Test 2:

A	B	fxn	Operation	Result
000101	000010	111	A-B	000011

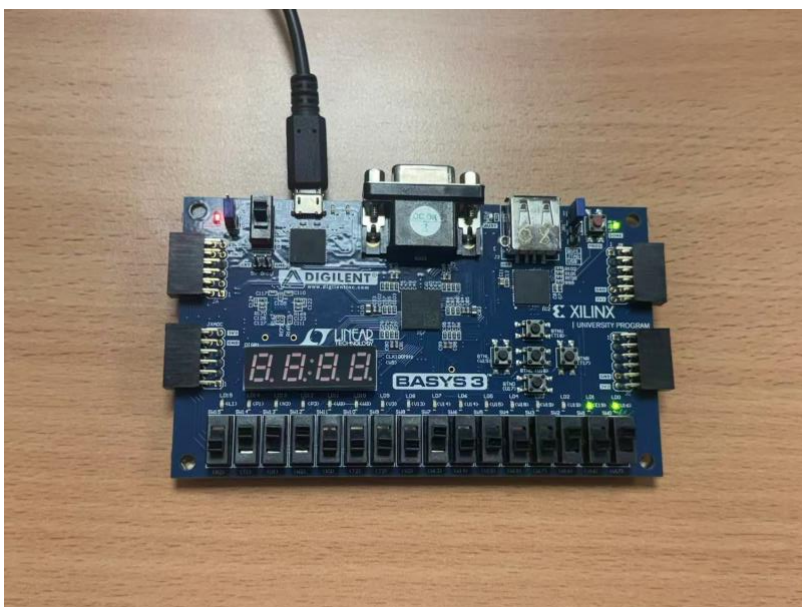


Figure 8: test 2

Test 3:

A	B	fxn	Operation	Result
101010	101010	101	A XNOR B	111111

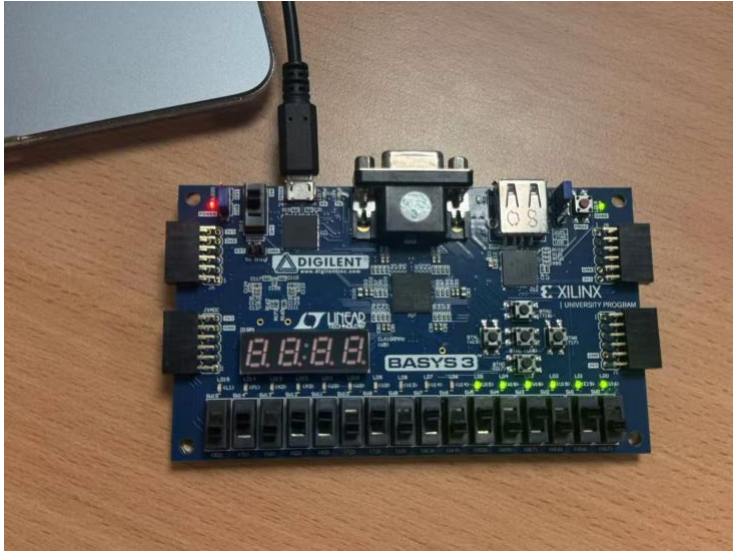


Figure 9: TEST 3

Test 4:

A	B	fxn	Operation	Result
000001	000010	100	A < B	000001

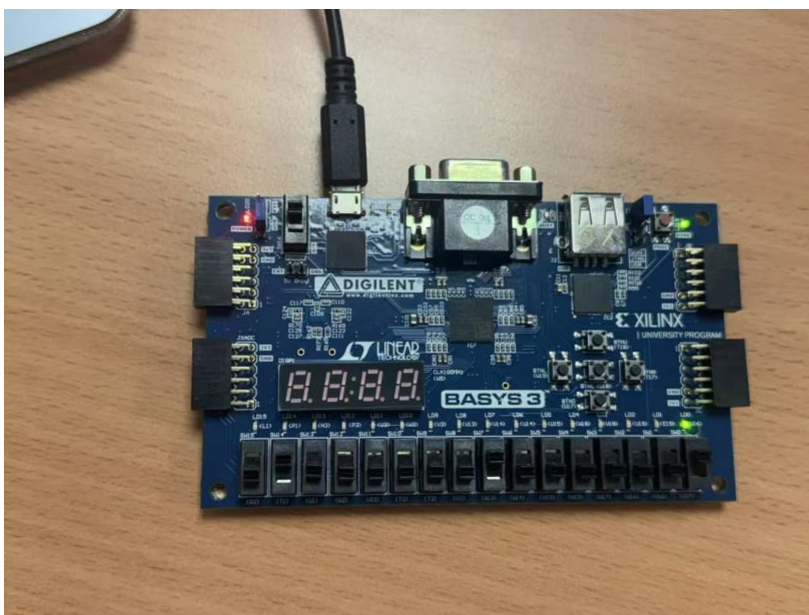


Figure 10: test 4

Test 5:

A	B	fxn	Operation	Result
000011	000000	000	Output A	000011

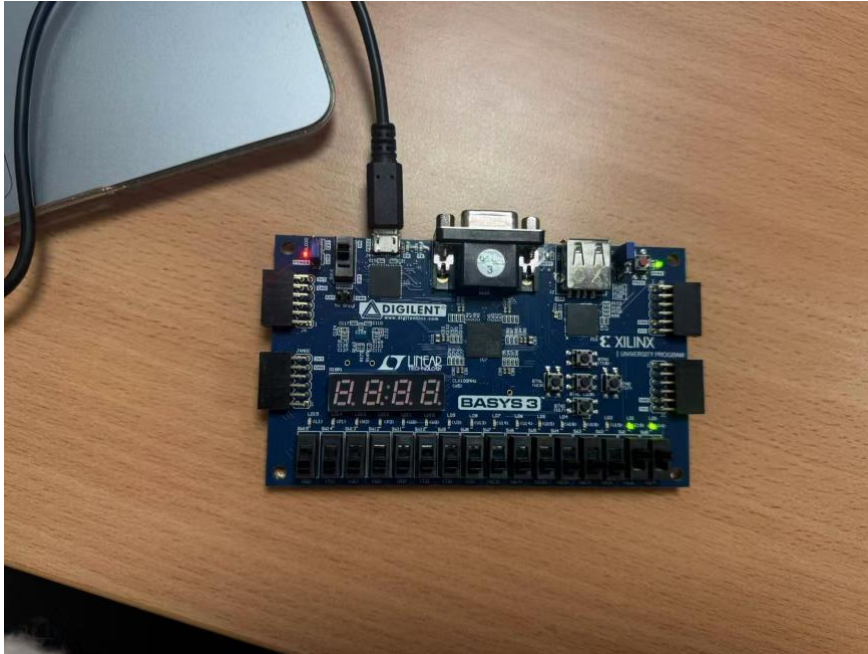


Figure 11: test 5

Test 6:

A	B	fxn	Operation	Result
000000	000010	001	Output B	000010

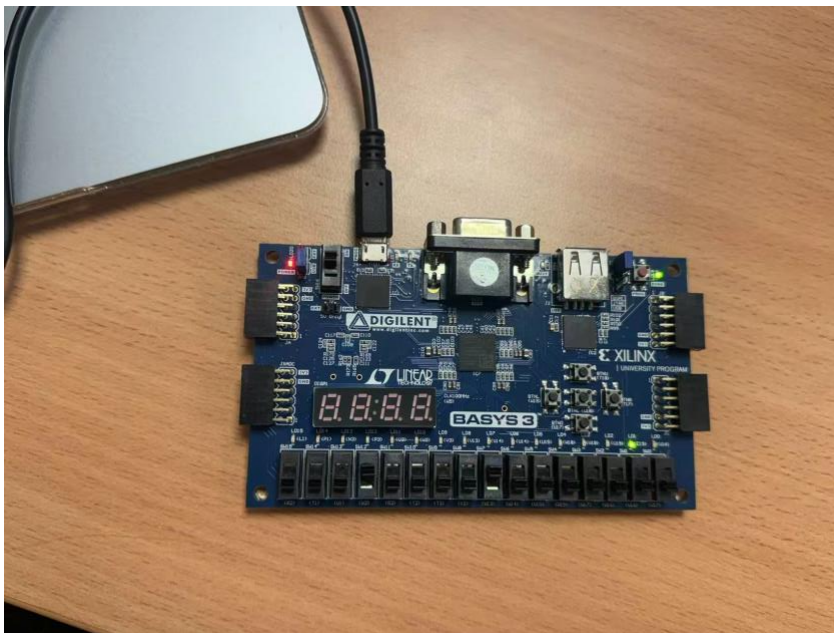


Figure 12: test 6.

Appendix – Lab E: Adder Implementation on Basys-3

Integration of Code

This design was built step by step using smaller modules, rather than one big one. Firstly, a 1-bit full adder was used, this module adds two input bits and a carry input, which produces a sum and carry output.

The full adder was reused multiple times inside a second module called sixbit ripple adder to create a 6 bit adder. In this design, the carry output from each stage is passed to the next stage.

A top level module called adder_top connects the design to the switches and LEDs onto the Basys-3 board. And the result of the operation is displayed onto the LEDs, together with the carry-out and overflow outputs. The .xdc file was used to map the FPGA pins to the physical switches and LEDs on the board.

Implementation Flow

This design was implemented using the standard flow in Vivado. First the Verilog source files were added to the project as design sources. A constraint file called adder.xdc was also included which connects the inputs and output of the design to the actual board we used.

Next, I synthesized the design, where Vivado automatically converts the Verilog code into logic gates that can run onto the FPGA. After that, the implementation stage placed and routed the logic elements onto the FPGA hardware.

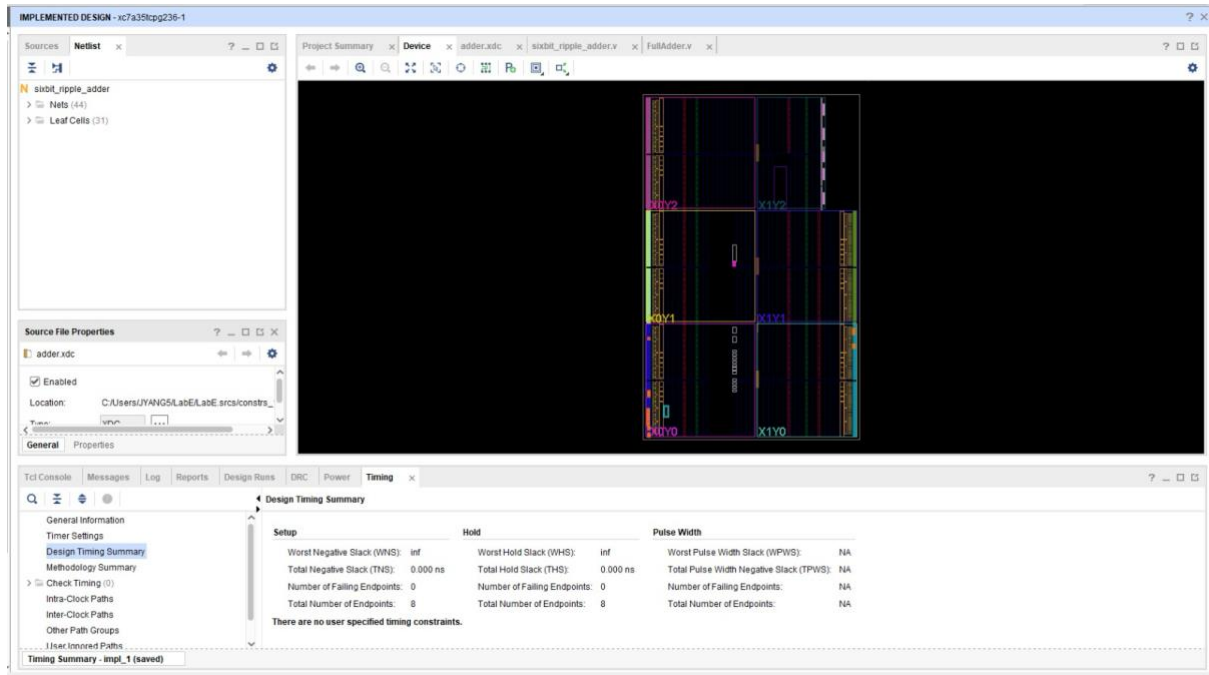


Figure 1: implementation view and timing summary of the 6 bit ripple carry adder after placement and routing in vivado. The timing report shows zero total negative slack, indicating the design meets timing requirements.

After everything was completed, Vivado generated a bitstream file. This was then programmed onto the Basy-3 board.

Observations and Resource Consumption

After implementation, the Vivado reports showed how many hardware resources were used by the design.

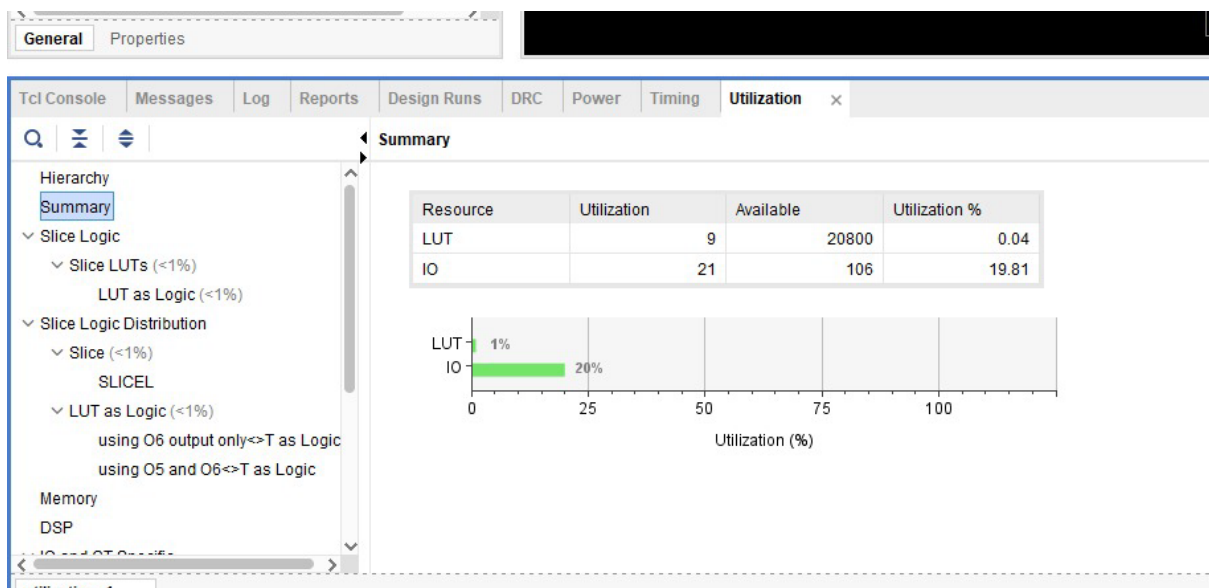


Figure 2: The utilization report showed that the circuit used 9 LUTs out of 20800 available, which is about 0.04% of the FPGA resources. This indicates that the adder uses only a very small amount of the FPGA's logic.

The design also used 21 I/O pins, which correspond to the switches used for inputs and the LEDs used for outputs.

Reports	Design Runs	DRC	Power	Timing	Utilization
Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	inf	Worst Hold Slack (WHS):	inf	Worst Pulse Width Slack (WPWS):	NA
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	NA
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	NA
Total Number of Endpoints:	8	Total Number of Endpoints:	8	Total Number of Endpoints:	NA
There are no user specified timing constraints.					
Timing Summary - timing_1					

Figure 3: The timing report showed no timing violations, and the total negative slack was **zero**, meaning the design meets the timing requirements and operates correctly on the FPGA.

During testing, changing the switch values changed the LED outputs accordingly. The LEDs correctly displayed the result of the addition or subtraction along with the carry-out and overflow signals.